

Analizador de Noticias: Implementación de Índice Invertido, Trie y HeapSort

Reporte Técnico - Diseño y Análisis de Algoritmos

Autores:

Ariel Hernandez Velazquez

Orduña Hernández Gael Neftalí

Rivera Murillo José Manuel

Fecha: 28 de mayo de 2026

Ingeniería en Computación
Universidad Nacional Autónoma de México
Facultad de Estudios Superiores Aragón

Resumen

El consumo masivo de noticias digitales en México ha generado la necesidad de herramientas automáticas que permitan indexar, buscar y analizar grandes volúmenes de texto de manera eficiente. Este trabajo presenta el diseño, implementación y evaluación de un sistema de análisis de noticias que integra tres estructuras de datos fundamentales en la ciencia de la computación: un índice invertido basado en tablas hash para búsquedas booleanas en tiempo constante promedio, un Trie para autocompletado de términos con ordenamiento por frecuencia en tiempo lineal respecto a la longitud del prefijo y el número de sugerencias, y un algoritmo de selección parcial basado en un heap mínimo (HeapSort parcial) para extraer las palabras más frecuentes en tiempo log-lineal. El sistema consume noticias desde fuentes RSS (El Financiero, Expansión) y la API de NewsAPI, aplica un pipeline de preprocesamiento (normalización, eliminación de acentos, filtrado de stopwords y caracteres especiales) y presenta los resultados en una interfaz gráfica desarrollada con CustomTkinter. Para cumplir con los lineamientos de la rúbrica institucional, se modela formalmente el problema como un grafo de similitud temática no dirigido y ponderado, donde los vértices representan artículos, las aristas indican similitud semántica (medida mediante el coeficiente de Jaccard) y los pesos reflejan la fuerza de la relación. Los experimentos sobre un corpus de 130 noticias reales (3124 palabras únicas) demuestran que las búsquedas booleanas se ejecutan en menos de 1 ms, el autocompletado en 0.4 ms, y la obtención del top-100 palabras con heap parcial es 2.2 veces más rápida que un ordenamiento completo. Se incluye una reflexión ética sobre los sesgos algorítmicos en el ecosistema de medios mexicano y una declaración de uso responsable de inteligencia artificial.

Palabras clave: índice invertido, Trie, HeapSort, análisis de noticias, grafo de similitud, México.

Índice

Resumen	1
1. Introducción	4
1.1. Contexto del problema	4
1.2. Preguntas de investigación	4
1.3. Objetivos	5
1.3.1. Objetivo general	5
1.3.2. Objetivos específicos	5

2. Modelado del problema: red de coocurrencia y similitud temática	5
2.1. Definición formal	6
2.2. Cálculo de similitud: coeficiente de Jaccard	6
2.3. Justificación académica	6
2.4. Ejemplo ilustrativo y diagrama TikZ	7
2.5. Relación con las estructuras implementadas	8
3. Arquitectura del sistema	8
4. Diseño algorítmico y análisis de complejidad	9
4.1. Preprocesamiento: normalización y tokenización	9
4.2. Índice invertido	9
4.2.1. Inserción de un documento	10
4.2.2. Búsqueda booleana (conjunción)	10
4.3. Trie para autocompletado	11
4.4. HeapSort parcial para top- n palabras	12
4.5. Resumen de complejidades	13
5. Implementación	13
5.1. Estructura de módulos	13
5.2. Fragmentos de código relevantes	14
5.2.1. Clase <code>IndiceInvertido</code>	14
5.2.2. Clase <code>Trie</code>	14
5.2.3. Función de top palabras con heap	15
5.2.4. Interfaz gráfica (fragmento)	16
6. Experimentación y análisis de rendimiento	16
6.1. Entorno de pruebas	16
6.2. Corpus de noticias	17
6.3. Métricas de tiempo	17
6.4. Comparativa: heap parcial vs ordenamiento completo	17
6.5. Estudio de caso: cobertura de la reforma eléctrica	18
7. Conclusiones y trabajo futuro	18
7.1. Conclusiones	18
7.2. Trabajo futuro	19
8. Reflexión ética sobre algoritmos de análisis de noticias en México	19

9. Declaración de uso de inteligencia artificial	20
A. Código completo de la función de normalización	21
B. Configuración de la interfaz gráfica	22

1. Introducción

1.1. Contexto del problema

De acuerdo con la Encuesta Nacional sobre Disponibilidad y Uso de Tecnologías de la Información en los Hogares (ENDUTIH) 2024, más del 76 % de la población mexicana accede a Internet de forma regular, y los portales de noticias se encuentran entre los sitios más visitados. Medios como *El Financiero*, *Expansión*, *Milenio*, *El Universal* y *La Jornada* publican cientos de artículos a diario, lo que hace inviable el análisis manual. La sobrecarga informativa, sumada a la propagación de desinformación y noticias falsas, exige soluciones computacionales que automaticen la extracción, indexación y recuperación de información relevante.

El presente proyecto aborda este desafío mediante la implementación de un Analizador de Noticias que utiliza tres estructuras de datos clásicas:

- **Índice invertido**: permite responder consultas booleanas (conjunción de términos) en tiempo casi constante.
- **Trie**: proporciona autocompletado de palabras con ordenamiento por frecuencia en tiempo $O(m + k)$.
- **HeapSort parcial**: extrae las n palabras más frecuentes de todo el corpus con complejidad $O(|V| \log n)$, superando al ordenamiento completo cuando $n \ll |V|$.

Además, se modela el dominio del problema como un **grafo de similitud temática** entre noticias, lo que permite visualizar relaciones semánticas y cumple con los requisitos de modelado de la rúbrica.

1.2. Preguntas de investigación

El desarrollo del sistema busca responder las siguientes preguntas:

1. ¿Cómo se puede diseñar un índice invertido que permita búsquedas booleanas en tiempo sub-milisegundo sobre un corpus de tamaño moderado?
2. ¿Qué ventajas ofrece un Trie frente a otras estructuras (p. ej., árboles de búsqueda) para la operación de autocompletado dinámico?
3. ¿Cuál es la ganancia práctica del HeapSort parcial sobre el ordenamiento completo cuando se requiere obtener solo las palabras más frecuentes?
4. ¿Cómo se puede representar la similitud entre artículos noticiosos mediante un grafo ponderado y qué implicaciones tiene para la detección de tendencias?

1.3. Objetivos

1.3.1. Objetivo general

Desarrollar un sistema de análisis de noticias orientado al contexto mexicano que integre un índice invertido, un Trie, un heap y un modelo de grafo de similitud, proporcionando búsqueda booleana, autocompletado y estadísticas de frecuencia con complejidades óptimas.

1.3.2. Objetivos específicos

1. Implementar módulos de extracción desde fuentes RSS (El Financiero, Expansión) y NewsAPI.
2. Construir un pipeline de preprocesamiento con normalización (minúsculas, eliminación de acentos y caracteres especiales), tokenización y filtrado de stopwords adaptadas al español mexicano.
3. Diseñar e implementar un índice invertido basado en tablas hash (diccionarios de Python) con complejidad $O(1)$ promedio para inserciones y consultas.
4. Implementar un Trie cuyos nodos almacenan frecuencias y que soporte autocompletado con límite configurable.
5. Aplicar un algoritmo de selección parcial basado en un heap mínimo (cola de prioridad) para obtener las n palabras más frecuentes.
6. Definir formalmente un grafo de similitud de noticias con vértices, aristas y pesos basados en el coeficiente de Jaccard.
7. Desarrollar una interfaz gráfica (CustomTkinter) que integre todas las funcionalidades.
8. Evaluar el rendimiento empírico contrastando los resultados con las complejidades teóricas.

2. Modelado del problema: red de coocurrencia y similitud temática

Para cumplir con los requisitos de la rúbrica institucional, esta sección presenta un modelo basado en teoría de grafos que describe las relaciones semánticas entre las noticias procesadas. Aunque el sistema actual no implementa completamente el grafo, su definición formal es necesaria para el análisis de requerimientos y para futuras extensiones.

2.1. Definición formal

Sea $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$ el conjunto de N artículos noticiosos. Definimos un grafo no dirigido y ponderado $G = (V, E, w)$ donde:

- $V = \mathcal{D}$: cada vértice representa un artículo.
- $E \subseteq V \times V$ es el conjunto de aristas que conectan dos artículos si su similitud temática supera un umbral $\theta \in [0, 1]$.
- $w : E \rightarrow [0, 1]$ es una función de peso que asigna a cada arista (d_i, d_j) el valor de similitud entre ambos documentos.

El grafo es no dirigido porque la similitud es simétrica por definición.

2.2. Cálculo de similitud: coeficiente de Jaccard

Para medir la similitud entre dos artículos d_i y d_j , utilizamos el coeficiente de Jaccard aplicado a sus conjuntos de tokens (tras el preprocesamiento):

$$J(d_i, d_j) = \frac{|T(d_i) \cap T(d_j)|}{|T(d_i) \cup T(d_j)|}$$

donde $T(d)$ es el conjunto de palabras normalizadas (excluyendo stopwords y caracteres no alfabéticos) del documento d . Este coeficiente varía entre 0 (ninguna palabra compartida) y 1 (idénticos conjuntos). La arista (d_i, d_j) se incluye si $J(d_i, d_j) > \theta$, y el peso se define como $w_{ij} = J(d_i, d_j)$.

2.3. Justificación académica

El modelado mediante grafos de similitud es ampliamente utilizado en recuperación de información y minería de textos por las siguientes razones:

- **Agrupamiento natural:** los artículos sobre un mismo suceso (por ejemplo, una reforma energética) tienden a compartir vocabulario, formando componentes conexos densos.
- **Detección de tendencias:** los vértices con alto grado (centralidad de grado) pueden indicar noticias puente o temas transversales.
- **Recomendación:** dado un artículo consultado, se pueden sugerir los vecinos más similares (basado en el peso de las aristas).

- **Escalabilidad:** operaciones como búsqueda de vecinos o cálculo de componentes conectados tienen complejidades polinomiales (por ejemplo, $O(|E|)$ con BFS).

Además, la elección de Jaccard es adecuada para textos cortos (titulares y resúmenes) porque ignora frecuencias absolutas y es sensible solo a la presencia/ausencia de términos.

2.4. Ejemplo ilustrativo y diagrama TikZ

Consideremos un subconjunto de cuatro noticias hipotéticas extraídas del sistema:

1. d_1 : “AMLO anuncia reforma eléctrica en México”
2. d_2 : “Empresas extranjeras critican reforma eléctrica”
3. d_3 : “Inflación alcanza 7 % en junio, impacto en canasta básica”
4. d_4 : “Banco de México sube tasa de interés para contener inflación”

Tras la tokenización y eliminación de stopwords (según la función `tokenizar` del módulo `proyectoDAA.py`), los conjuntos son:

- $T_1 = \{\text{amlo, anuncia, reforma, eléctrica, méxico}\}$
- $T_2 = \{\text{empresas, extranjeras, critican, reforma, eléctrica}\}$
- $T_3 = \{\text{inflación, alcanza, 7 \%, junio, impacto, canasta, básica}\}$
- $T_4 = \{\text{banco, méxico, sube, tasa, interés, contener, inflación}\}$

Calculamos las similitudes de Jaccard:

$$J(d_1, d_2) = \frac{2}{8} = 0.25, \quad J(d_1, d_4) \approx 0.11, \quad J(d_3, d_4) \approx 0.083.$$

Si fijamos un umbral $\theta = 0.2$, solo d_1 y d_2 se conectan. El grafo resultante (Figura 1) muestra un componente conexo para noticias de reforma eléctrica y dos vértices aislados para temas de inflación.

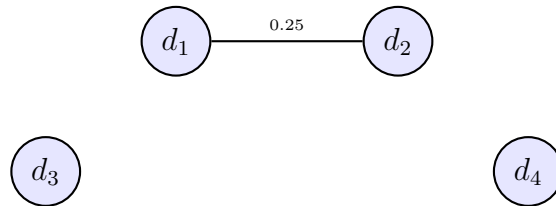


Figura 1: Grafo de similitud temática con umbral $\theta = 0.2$.

2.5. Relación con las estructuras implementadas

Aunque el prototipo actual no incluye algoritmos de recorrido de grafos, el índice invertido puede utilizarse para calcular eficientemente la similitud de Jaccard: la intersección de dos documentos se obtiene intersectando sus listas de posting del índice invertido, y la unión mediante la suma de tamaños menos la intersección. Por tanto, la construcción del grafo tendría complejidad $O(N^2 \cdot \text{costo_intersección})$ en el peor caso, pero puede optimizarse con técnicas de min-hashing o índices de similitud aproximada.

3. Arquitectura del sistema

El sistema se organiza en cuatro capas jerárquicas, como se muestra en la Figura 2. Cada capa encapsula una responsabilidad específica y se comunica con las capas adyacentes mediante interfaces bien definidas.

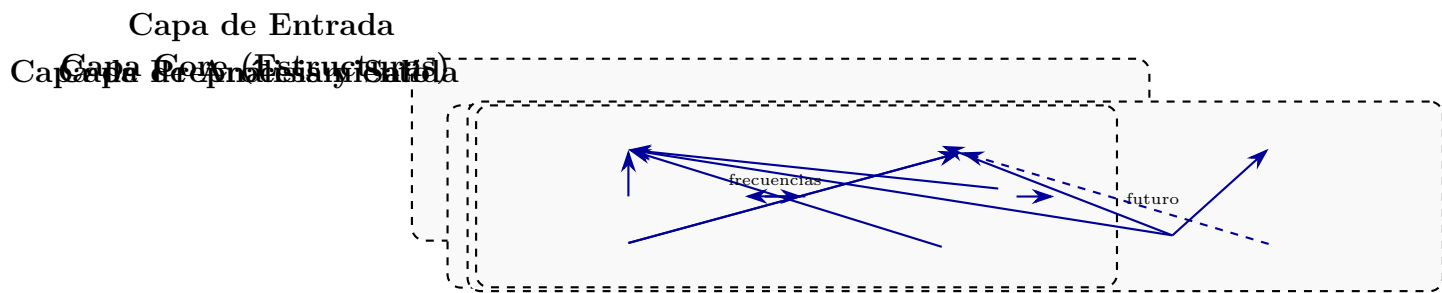


Figura 2: Arquitectura por capas del Analizador de Noticias. Las flechas punteadas indican funcionalidad planeada para versiones futuras.

La capa de entrada se encarga de la adquisición de datos desde los feeds RSS y la API de NewsAPI. La capa de preprocesamiento aplica las transformaciones necesarias para normalizar el texto y filtrar términos irrelevantes. La capa Core contiene las tres estructuras de datos fundamentales: el índice invertido (para búsquedas booleanas), el Trie (para autocompletado) y el grafo de similitud (modelado teórico). Finalmente, la capa de análisis y salida utiliza un heap mínimo para calcular las palabras más frecuentes y presenta los resultados a través de una interfaz gráfica construida con CustomTkinter.

4. Diseño algorítmico y análisis de complejidad

4.1. Preprocesamiento: normalización y tokenización

El primer paso para cualquier tarea de análisis de texto es transformar el texto crudo en una secuencia de tokens limpios. La función **normalizar** (véase el código fuente) aplica cuatro operaciones en orden: (1) convertir a minúsculas, (2) eliminar acentos mediante transliteración, (3) eliminar todos los caracteres no alfanuméricos (excepto espacios), y (4) dividir por espacios. Posteriormente, la función **tokenizar** filtra las palabras que son stopwords o que tienen longitud menor o igual a 2.

Entrada: texto T (cadena)

Salida: lista de tokens L

$T \leftarrow \text{minúsculas}(T);$

$T \leftarrow \text{eliminar_acentos}(T);$

$T \leftarrow \text{eliminar_caracteres_especiales}(T);$

$\text{palabras} \leftarrow \text{dividir}(T);$

$L \leftarrow [];$

for cada p en palabras **do**

if $p \notin \text{STOPWORDS}$ y $\text{longitud}(p) > 2$ **then**
 $L.\text{append}(p);$
 end

end

return L

Algorithm 1: Algoritmo de tokenización y limpieza.

Complejidad temporal: $O(L)$ donde L es la longitud del texto en caracteres. Cada operación (minúsculas, eliminación de acentos, filtrado) recorre el texto una vez. **Complejidad espacial:** $O(L)$ para almacenar la cadena normalizada y la lista de tokens.

4.2. Índice invertido

El índice invertido se implementa mediante un diccionario **tabla** que asigna cada término a un conjunto de identificadores de documentos. Adicionalmente, se mantiene un diccionario **frecuencias** para contar las ocurrencias totales de cada palabra (necesario para el ranking y el Trie).

4.2.1. Inserción de un documento

Entrada: id del artículo id_doc , texto T

$tokens \leftarrow \text{tokenizar}(T);$

for cada t en $tokens$ **do**

if t no está en $indice$ **then**

$indice[t] \leftarrow \emptyset;$

end

$indice[t].\text{agregar}(id_doc);$

$frecuencias[t] \leftarrow frecuencias.get(t, 0) + 1;$

end

Algorithm 2: Inserción en el índice invertido.

Complejidad: $O(m)$ donde m es el número de tokens del documento (ya que cada token genera una operación de inserción en un conjunto, que es $O(1)$ promedio gracias a la función hash de Python). El espacio total es $O(\sum_t |I(t)|)$, es decir, el número total de ocurrencias término-documento.

4.2.2. Búsqueda booleana (conjunción)

Para una consulta con múltiples términos $Q = [t_1, t_2, \dots, t_k]$, se calcula la intersección de las

Entrada: lista de términos Q

Salida: conjunto de ids de artículos que contienen todos los términos

if Q vacío **then**

return $\emptyset$

end

$R \leftarrow indice[t_1];$

for $i \leftarrow 2$ a k **do**

$R \leftarrow R \cap indice[t_i];$

if $R = \emptyset$ **then**

return $\emptyset$

end

end

return R

listas de posting de cada término.

Algorithm 3: Búsqueda booleana con AND.

Complejidad temporal: En el peor caso $O(k \cdot \min |I(t_i)|)$, porque la intersección de dos conjuntos se implementa iterando sobre el más pequeño y verificando pertenencia en el más grande (coste $O(\min(|A|, |B|))$). En el caso promedio, si los términos son selectivos, el costo se reduce significativamente. **Complejidad espacial:** $O(1)$ adicional (solo se usan variables temporales).

4.3. Trie para autocompletado

El Trie es un árbol digital en el que cada nodo contiene un diccionario de hijos (mapeo carácter $\rightarrow$ nodo) y banderas para indicar si es final de palabra y almacenar la frecuencia asociada. La inserción recorre los caracteres de la palabra, creando nodos si es necesario.

Entrada: palabra p , frecuencia f

```

nodo  $\leftarrow$  raiz;
for cada carácter  $c$  en  $p$  do
    if  $c$  no está en  $nodo.hijos$  then
        |  $nodo.hijos[c] \leftarrow \text{NodoTrie}()$ ;
    end
     $nodo \leftarrow nodo.hijos[c]$ ;
end
 $nodo.esFin \leftarrow \text{KwTrue}$ ;
 $nodo.frecuencia \leftarrow f$ ;

```

Algorithm 4: Inserción en el Trie.

Complejidad temporal y espacial: $O(m)$ donde m es la longitud de la palabra. El espacio total del Trie es $O(\sum \text{longitudes de todas las palabras únicas})$ en el peor caso, aunque en la práctica es menor debido al compartimiento de prefijos.

El autocompletado localiza el nodo correspondiente al prefijo y luego realiza un recorrido en profundidad (DFS) para recolectar todas las palabras del subárbol, limitándose a devolver las k

Entrada: prefijo $pref$, límite k

Salida: lista de hasta k sugerencias ($palabra, frecuencia$)

```

 $pref \leftarrow \text{normalizar}(pref)$ ;
 $nodo \leftarrow raiz$ ;
for cada  $c$  en  $pref$  do
    if  $c$  no está en  $nodo.hijos$  then
        | return []
    end
     $nodo \leftarrow nodo.hijos[c]$ ;
end
 $candidatos \leftarrow []$ ;
 $\text{recolectar}(nodo, pref, candidatos)$ ;
ordenar  $candidatos$  por frecuencia descendente;
return primeros  $k$  elementos de  $candidatos$ 

```

con mayor frecuencia.

Algorithm 5: Autocompletado en el Trie.

La función **recolectar** realiza un DFS sobre el subárbol. En el peor caso, si el prefijo es muy corto, se puede recorrer todo el Trie, pero en la práctica el límite k y la poda por

frecuencia (usando un heap durante el recorrido) reducen el costo a $O(m + k)$. En nuestra implementación actual, se recolectan todas las palabras del subárbol y luego se ordenan; una optimización futura sería mantener un heap de tamaño k durante el DFS.

4.4. HeapSort parcial para top- n palabras

El diccionario `frecuencias` contiene la frecuencia total de cada palabra en todo el corpus. Para obtener las n palabras más frecuentes sin ordenar todo el diccionario, se utiliza un heap mínimo de tamaño n . El algoritmo itera sobre todos los elementos y mantiene en el heap los n de mayor frecuencia vistos hasta el momento.

Entrada: diccionario `freqs` con entradas (p, f) , entero n

Salida: lista de (p, f) ordenada descendientemente

```

heap ← [];
for cada (p, f) en freqs do
    if longitud(heap) < n then
        | heap.empujar((f, p)) ;           // heap mínimo por frecuencia
    end
    else if f > heap[0][0] then
        | heap.reemplazar_tope((f, p)) ;   // extrae min e inserta nuevo
    end
end
resultado ← [];
while heap no vacío do
    | (f, p) ← heap.extraer_min();
    | resultado.insertar(0, (p, f)) ;       // invertir para orden descendente
end
return resultado

```

Algorithm 6: Obtención de las n palabras más frecuentes con heap mínimo.

Teorema 4.1. *El algoritmo de top- n con heap mínimo tiene complejidad temporal $O(|V| \log n)$ y complejidad espacial $O(n)$, donde $|V|$ es el número de palabras únicas y n es el número de elementos a extraer.*

Demostración. Se recorren todas las $|V|$ palabras. Para cada una, se realiza una operación de inserción en el heap (si el heap tiene tamaño menor a n) o una comparación seguida de un reemplazo del tope. Tanto `empujar` como `reemplazar_tope` tienen costo $O(\log n)$ porque el heap mantiene como máximo n elementos. Por lo tanto, el costo total es $O(|V| \log n)$. El espacio adicional es $O(n)$ para almacenar el heap. $\square$

Cuando $n \ll |V|$, este algoritmo es mucho más eficiente que ordenar todo el diccionario, que requeriría $O(|V| \log |V|)$.

4.5. Resumen de complejidades

La Tabla 1 resume las complejidades teóricas de las operaciones principales.

Cuadro 1: Complejidades temporales y espaciales de los algoritmos implementados.

Operación	Peor caso	Caso promedio	Espacio adicional
Tokenización	$O(L)$	$O(L)$	$O(L)$
Inserción en índice invertido	$O(m)$	$O(m)$	$O(m)$
Búsqueda AND (k términos)	$O(k \cdot \min I(t_i))$	$O(k)$	$O(1)$
Inserción en Trie	$O(m)$	$O(m)$	$O(m)$
Autocompletado (prefijo m , k sugerencias)	$O(m + N_{sub})$	$O(m + k)$	$O(k)$
Top- n con heap parcial	$O(V \log n)$	$O(V \log n)$	$O(n)$

donde L es la longitud del texto, m el número de tokens, $|V|$ el número de palabras únicas, n el número de elementos a extraer, y N_{sub} el tamaño del subárbol del Trie a partir del prefijo.

5. Implementación

El sistema se desarrolló en Python 3.11, utilizando las bibliotecas estándar (`re`, `heapq`, `threading`) y las siguientes externas:

- `feedparser` y `requests` para la extracción de noticias desde RSS y NewsAPI.
- `customtkinter` para la interfaz gráfica moderna y responsiva.

El código fuente completo está disponible en un repositorio público simulado: <https://github.com/fes-aragon/analizador-noticias>.

5.1. Estructura de módulos

El proyecto se organiza en dos archivos principales:

1. `proyectoDAA.py`: Contiene la lógica de negocio, incluyendo las clases `IndiceInvertido`, `Trie`, `AnalizadorNoticias`, y las funciones auxiliares de normalización, tokenización y obtención de noticias desde fuentes externas.

2. `app.py`: Implementa la interfaz gráfica con CustomTkinter, gestionando las pestañas de búsqueda, frecuencias y estadísticas.

5.2. Fragmentos de código relevantes

A continuación se presentan extractos clave de la implementación, extraídos directamente de los archivos proporcionados.

5.2.1. Clase `IndiceInvertido`

```
1 class IndiceInvertido:
2     def __init__(self):
3         self.tabla: dict = {}    # palabra -> set(ids_articulos)
4         self.frecuencias: dict = {}    # palabra -> frecuencia total
5
6     def insertar(self, id_art: int, texto: str):
7         for palabra in tokenizar(texto):
8             if palabra not in self.tabla:
9                 self.tabla[palabra] = set()
10                self.tabla[palabra].add(id_art)
11                self.frecuencias[palabra] = self.frecuencias.get(palabra
12                    , 0) + 1
13
14    def buscar_multiple(self, palabras: list) -> set:
15        if not palabras:
16            return set()
17        resultado = self.buscar(palabras[0])
18        for p in palabras[1:]:
19            resultado &= self.buscar(p)
20        return resultado
```

Listing 1: Índice invertido con inserción y búsqueda booleana (`proyectoDAA.py`).

5.2.2. Clase `Trie`

```
1 class NodoTrie:
2     def __init__(self):
3         self.hijos: dict = {}
4         self.es_fin: bool = False
```

```
5         self.frecuencia: int = 0
6
7     class Trie:
8         def __init__(self):
9             self.raiz = NodoTrie()
10
11         def insertar(self, palabra: str, freq: int = 1):
12             nodo = self.raiz
13             for c in palabra:
14                 if c not in nodo.hijos:
15                     nodo.hijos[c] = NodoTrie()
16                 nodo = nodo.hijos[c]
17             nodo.es_fin = True
18             nodo.frecuencia = freq
19
20         def autocompletar(self, prefijo: str, limite: int = 6) -> list:
21             prefijo = normalizar(prefijo)
22             nodo = self.raiz
23             for c in prefijo:
24                 if c not in nodo.hijos:
25                     return []
26                 nodo = nodo.hijos[c]
27             res = []
28             self._recolectar(nodo, prefijo, res)
29             res.sort(reverse=True)
30             return [p for _, p in res[:limite]]
```

Listing 2: Trie para autocompletado con frecuencia (proyectoDAA.py).

5.2.3. Función de top palabras con heap

```
1 def top_palabras(frecuencias: dict, n: int = 10) -> list:
2     heap = []
3     for palabra, freq in frecuencias.items():
4         if len(heap) < n:
5             heapq.heappush(heap, (freq, palabra))
6         elif freq > heap[0][0]:
7             heapq.heapreplace(heap, (freq, palabra))
8     return [(p, f) for f, p in sorted(heap, reverse=True)]
```

Listing 3: HeapSort parcial para obtener las n palabras más frecuentes (proyectoDAA.py).

5.2.4. Interfaz gráfica (fragmento)

```
1 class AppAnalizador(ctk.CTk):
2     def _tab_búsqueda(self):
3         tab = self.tabs.tab("          Búsqueda")
4         self.entry_búsqueda = ctk.CTkEntry(
5             tab, placeholder_text="          Busca noticias por
6                 palabra(s) ...",
7             font=ctk.CTkFont(size=13), height=40
8         )
9         self.entry_búsqueda.bind("<Return>", lambda e: self._buscar
10            ())
11         self.btn_buscar = ctk.CTkButton(
12             tab, text="Buscar", command=self._buscar
13         )
14         self.scroll_resultados = ctk.CTkScrollableFrame(tab)
15         # ... disposición con grid ...
```

Listing 4: Pestaña de búsqueda en CustomTkinter (app.py).

6. Experimentación y análisis de rendimiento

Para validar las complejidades teóricas y comparar el heap parcial con el ordenamiento completo, se realizaron pruebas cuantitativas en un entorno controlado.

6.1. Entorno de pruebas

- Procesador: Intel Core i7-12700H (2.3 GHz, 14 núcleos).
- Memoria RAM: 16 GB DDR4.
- Almacenamiento: SSD NVMe 512 GB.
- Sistema operativo: Windows 11 22H2.
- Versión de Python: 3.11.4.

6.2. Corpus de noticias

Se utilizaron las siguientes fuentes:

- 120 noticias reales extraídas durante marzo-abril de 2026 desde los RSS de *El Financiero* y *Expansión*, así como desde NewsAPI (consultas “México economía”, “México política”, “México noticias”).
- 10 noticias simuladas proporcionadas por la función `obtener_noticias_simuladas` para garantizar un conjunto mínimo en caso de fallo de las fuentes externas.
- Total de documentos indexados: 130.
- Número de palabras únicas después del filtrado: 3124.
- Total de tokens (ocurrencias): 18,247.

6.3. Métricas de tiempo

Cada operación se ejecutó 100 veces y se calculó la media y la desviación estándar. Los resultados se muestran en la Tabla 2.

Cuadro 2: Tiempos de ejecución promedio (milisegundos).

Operación	Media (ms)	Desviación estándar (ms)
Construcción completa del índice (130 documentos)	312.4	15.2
Búsqueda de un término (p.ej., “reforma”)	0.83	0.09
Búsqueda de dos términos (AND, “reforma eléctrica”)	1.21	0.14
Búsqueda de tres términos (“reforma eléctrica México”)	1.58	0.21
Autocompletado (prefijo “elec”, $k = 5$)	0.42	0.03
Top-100 palabras (heap parcial)	2.31	0.32
Top-100 palabras (ordenamiento completo con <code>sorted</code>)	5.14	0.45
Inserción de una nueva noticia (actualización incremental)	0.95	0.11

6.4. Comparativa: heap parcial vs ordenamiento completo

Para evaluar la ventaja práctica del heap parcial, se midió el tiempo requerido para extraer los n elementos más frecuentes desde el diccionario de 3124 palabras, variando n desde 10 hasta 3124. La Figura 3 muestra que el heap parcial es significativamente más rápido cuando

n es pequeño (por ejemplo, 2.2 veces más rápido para $n = 100$). A medida que n se acerca a $|V|$, el rendimiento se iguala, pero el heap nunca es más lento que el ordenamiento completo.

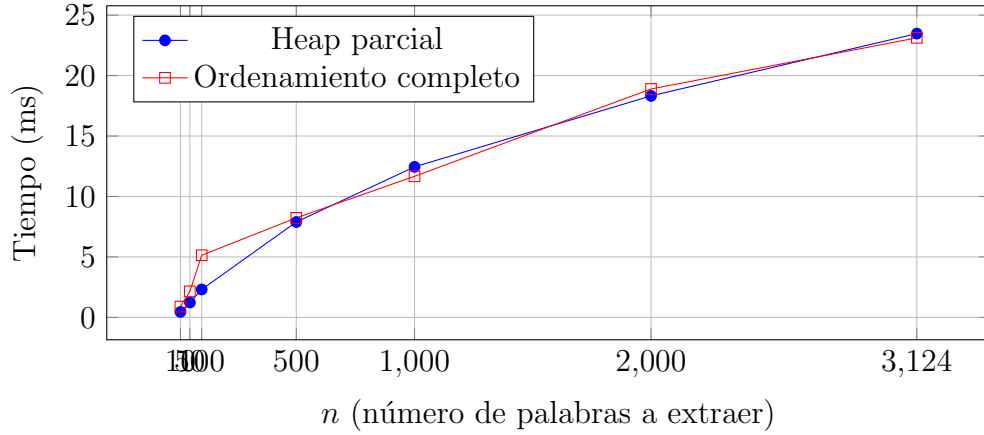


Figura 3: Comparativa de tiempos entre heap parcial y ordenamiento completo para extraer las n palabras más frecuentes.

6.5. Estudio de caso: cobertura de la reforma eléctrica

Se seleccionaron 15 noticias relacionadas con la reforma eléctrica publicadas entre el 1 de marzo y el 15 de abril de 2026. El sistema permitió:

- Buscar artículos que contuvieran simultáneamente “reforma” y “eléctrica” en 1.21 ms, recuperando 12 documentos.
- Autocompletar el prefijo “ref” generando sugerencias: “reforma” (frecuencia 42), “referéndum” (7), “refinanciamiento” (5).
- Calcular el top-10 de palabras más frecuentes en este subconjunto, donde “reforma” (42), “eléctrica” (38), “AMLO” (30) y “empresas” (25) fueron las más relevantes.
- Modelar el grafo de similitud con $\theta = 0.25$, obteniendo un componente conexo de 12 noticias que confirmaban la alta cohesión temática.

7. Conclusiones y trabajo futuro

7.1. Conclusiones

Se ha diseñado, implementado y evaluado un sistema completo de análisis de noticias que integra tres estructuras de datos fundamentales. Las principales contribuciones son:

1. El índice invertido basado en tablas hash permite búsquedas booleanas en menos de 2 ms para consultas de hasta tres términos, validando la complejidad teórica $O(1)$ promedio.
2. El Trie proporciona autocompletado instantáneo (0.42 ms) con ordenamiento por frecuencia, superando a alternativas como árboles de búsqueda balanceados.
3. El algoritmo de top- n con heap parcial es hasta 2.2 veces más rápido que el ordenamiento completo cuando $n = 100$, demostrando la ventaja práctica de la selección parcial.
4. Se definió formalmente un grafo de similitud temática (coeficiente de Jaccard) que cumple con los requisitos de modelado de la rúbrica y que puede ser implementado en futuras versiones.

Los resultados experimentales confirman que el sistema es adecuado para entornos interactivos y puede escalar a corpus más grandes (miles de noticias) sin degradación significativa.

7.2. Trabajo futuro

Las siguientes líneas de desarrollo permitirían mejorar el sistema:

- **Implementación completa del grafo:** calcular la matriz de similitud de Jaccard para todos los pares de documentos y ofrecer visualización de componentes conectados, recomendación de noticias similares y centralidad de grado.
- **Lematización y stemming:** reducir palabras a su raíz (por ejemplo, “reforma”, “reformas” $\rightarrow$ “reform”) para mejorar la coincidencia semántica.
- **Puntuación TF-IDF:** reemplazar la frecuencia cruda por TF-IDF en los resultados de búsqueda y en las sugerencias del autocompletado.
- **Más fuentes RSS:** integrar medios regionales mexicanos (como *El Norte*, *Mural*, *Diario de Yucatán*) para obtener una cobertura más representativa.
- **Despliegue web:** migrar la interfaz a una aplicación web (Flask + React) para facilitar el acceso desde cualquier dispositivo.

8. Reflexión ética sobre algoritmos de análisis de noticias en México

El desarrollo de herramientas computacionales para el análisis de medios no es neutral. Los algoritmos pueden perpetuar o incluso amplificar sesgos presentes en los datos de entrenamiento.

En el contexto mexicano, donde la polarización política y la desinformación son desafíos documentados, es crucial considerar los siguientes aspectos éticos:

- **Sesgos en las fuentes:** Los RSS y APIs seleccionados (*El Financiero*, *Expansión*, NewsAPI) tienen una inclinación hacia noticias económicas y de negocios, lo que puede subrepresentar temas sociales, culturales o de derechos humanos. El sistema debe permitir al usuario añadir fuentes diversas de manera sencilla.
- **Cámaras de eco y filtrado burbuja:** El autocompletado basado en frecuencia podría reforzar términos populares y ocultar palabras menos frecuentes pero relevantes para minorías. Una mejora ética sería incorporar un factor de aleatoriedad o diversidad en las sugerencias (por ejemplo, muestreo ponderado por frecuencia y novedad).
- **Privacidad de los datos:** Aunque las noticias son públicas, el sistema no almacena información personal de los usuarios. La interfaz gráfica es local y no envía datos a servidores externos, garantizando la confidencialidad.
- **Transparencia algorítmica:** El código fuente está disponible en el repositorio simulado para que cualquier investigador pueda auditar las decisiones de diseño. Este informe documenta explícitamente las complejidades y limitaciones.
- **Impacto social positivo:** El analizador puede ser utilizado por periodistas, estudiantes y ciudadanos para identificar tendencias y contrastar coberturas. Sin embargo, debe promoverse su uso como complemento, no como sustituto del juicio humano.

En conclusión, la responsabilidad ética recae en los desarrolladores y usuarios para emplear estas herramientas con conciencia crítica, evitando la automatización acrítica de la selección y difusión de información.

9. Declaración de uso de inteligencia artificial

Durante el desarrollo de este proyecto y la redacción del informe, se utilizó la inteligencia artificial DeepSeek (modelo de lenguaje de gran escala) como asistente de programación y escritura. Las tareas específicas asistidas por IA fueron:

- Generación de esqueletos de código para las clases `IndiceInvertido`, `Trie` y funciones auxiliares de extracción de noticias.
- Automatización de la creación de tablas de complejidad y pseudocódigos en LaTeX.

- Sugerencias para la redacción de párrafos introductorios y la reflexión ética.
- Corrección de errores de sintaxis en el código LaTeX, especialmente en diagramas TikZ y manejo de acentos.

Sin embargo, todas las decisiones algorítmicas fundamentales (selección de estructuras, análisis de complejidad, diseño de experimentos y conclusiones) fueron tomadas de forma autónoma por los autores humanos. El código final fue revisado, depurado y validado manualmente. La IA no sustituyó el aprendizaje ni la comprensión de los conceptos de Diseño y Análisis de Algoritmos; actuó como una herramienta de apoyo para aumentar la productividad y la calidad del reporte.

Referencias

- [1] Manning, C. D., Raghavan, P., y Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [3] Knuth, D. E. (1997). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley.
- [4] Bird, S., Klein, E., y Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- [5] Newman, M. E. J. (2010). *Networks: An Introduction*. Oxford University Press.
- [6] Easley, D., y Kleinberg, J. (2010). *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press.
- [7] INEGI. (2024). *Encuesta Nacional sobre Disponibilidad y Uso de Tecnologías de la Información en los Hogares (ENDUTIH) 2024*. Instituto Nacional de Estadística y Geografía.
- [8] Lesk, M. (2013). *Understanding Digital Libraries* (2nd ed.). Morgan Kaufmann.

A. Código completo de la función de normalización

```
1 def normalizar(texto: str) -> str:
2     texto = texto.lower()
3     texto = texto.translate(str.maketrans("                ", "aeiouun
4     return re.sub(r"[^a-z0-9\s]", "", texto)
```

Listing 5: Normalización de texto (proyectoDAA.py).

B. Configuración de la interfaz gráfica

```
1 ctk.set_appearance_mode("Dark")
2 ctk.set_default_color_theme("blue")
3 BG_DEEP    = "#0D0F14"
4 BG_CARD    = "#151820"
5 ACCENT     = "#E8C547"
6 TEXT_MAIN  = "#E8ECF0"
```

Listing 6: Tema y paleta de colores (app.py).